

SQL SUBQUERIES – MULTI-LEVEL LAB

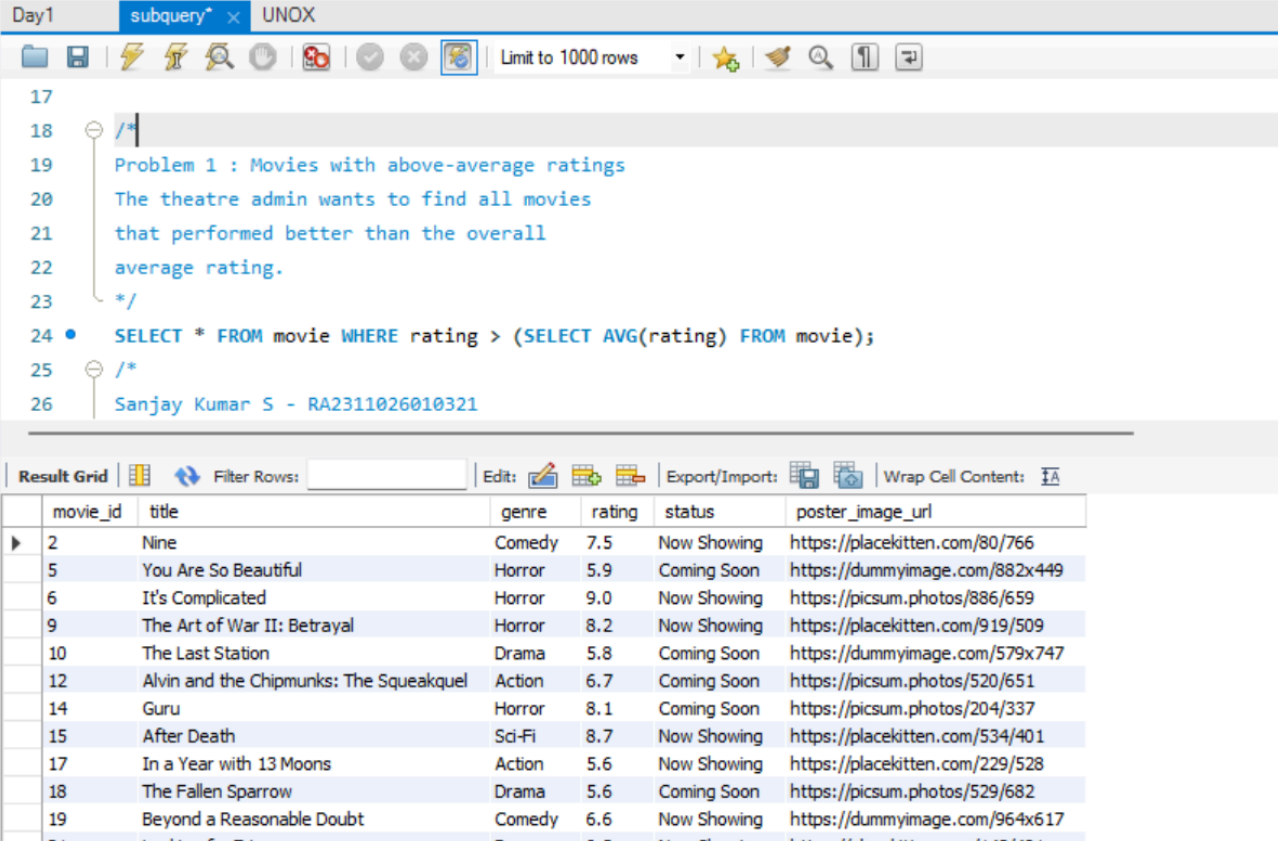
LEVEL 1 – Basic Subqueries

Problem 1: Movies with above-average ratings

Complexity: **Easy**

Scenario:

The theatre admin wants to find all movies that performed better than the overall average rating.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL query:

```

SELECT * FROM movie WHERE rating > (SELECT AVG(rating) FROM movie);

```

The result grid displays the following data:

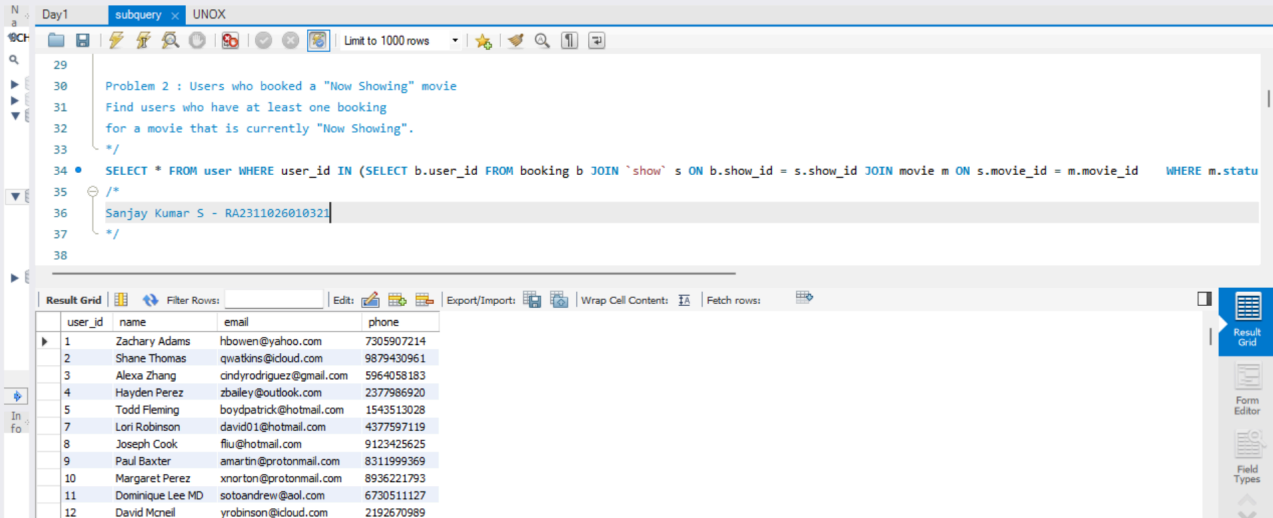
movie_id	title	genre	rating	status	poster_image_url
2	Nine	Comedy	7.5	Now Showing	https://placekitten.com/80/766
5	You Are So Beautiful	Horror	5.9	Coming Soon	https://dummyimage.com/882x449
6	It's Complicated	Horror	9.0	Now Showing	https://picsum.photos/886/659
9	The Art of War II: Betrayal	Horror	8.2	Now Showing	https://placekitten.com/919/509
10	The Last Station	Drama	5.8	Coming Soon	https://dummyimage.com/579x747
12	Alvin and the Chipmunks: The Squeakquel	Action	6.7	Coming Soon	https://picsum.photos/520/651
14	Guru	Horror	8.1	Coming Soon	https://picsum.photos/204/337
15	After Death	Sci-Fi	8.7	Now Showing	https://placekitten.com/534/401
17	In a Year with 13 Moons	Action	5.6	Now Showing	https://placekitten.com/229/528
18	The Fallen Sparrow	Drama	5.6	Coming Soon	https://picsum.photos/529/682
19	Beyond a Reasonable Doubt	Comedy	6.6	Now Showing	https://dummyimage.com/964x617

Problem 2: Users who booked a “Now Showing” movie

Complexity: **Medium**

Scenario:

Find users who have at least one booking for a movie that is currently “Now Showing”.



The screenshot shows a database query editor interface. The top pane contains a SQL query to find users who have booked a "Now Showing" movie. The bottom pane displays the results in a table format.

SQL Query:

```
SELECT * FROM user WHERE user_id IN (SELECT b.user_id FROM booking b JOIN 'show' s ON b.show_id = s.show_id JOIN movie m ON s.movie_id = m.movie_id WHERE m.status = 'Now Showing')
```

Results Table:

user_id	name	email	phone
1	Zachary Adams	hbowen@yahoo.com	7305907214
2	Shane Thomas	qwatkins@icloud.com	9879430961
3	Alexa Zhang	cindyrodriguez@gmail.com	5964058183
4	Hayden Perez	zbailey@outlook.com	2377986920
5	Todd Fleming	boydpatrick@hotmail.com	1543513028
7	Lori Robinson	david01@hotmail.com	4377597119
8	Joseph Cook	flu@hotmail.com	9123425625
9	Paul Baxter	amartin@protonmail.com	8311999369
10	Margaret Perez	xnorton@protonmail.com	8936221793
11	Dominique Lee MD	sotoandrew@aol.com	6730511127
12	David Mcnell	yrobinson@icloud.com	2192670989

SQL Solution:

Problem 3 (Higher-order): Movies with more reviews than any Action movie

Complexity: **Hard**

Scenario:

Management wants to see which movies have equal or more reviews than *any* Action movie.

The screenshot shows a SQL query editor with the following SQL query:

```
/*
Problem 3 : Movies with more reviews than any
Action movie
Management wants to see which movies have
equal or more reviews than any Action movie.
*/
SELECT movie_id, COUNT(*) AS review_count FROM review GROUP BY movie_id HAVING COUNT(*) >= ANY (SELECT COUNT(*) FROM review r JOIN movie m ON r.movie_id = m.movie_id WHERE m.genre = 'Action')
```

The result grid shows the following data:

movie_id	review_count
1	79
2	38
3	63
4	21
5	78
6	42
7	99
8	70
9	50
10	73
11	37

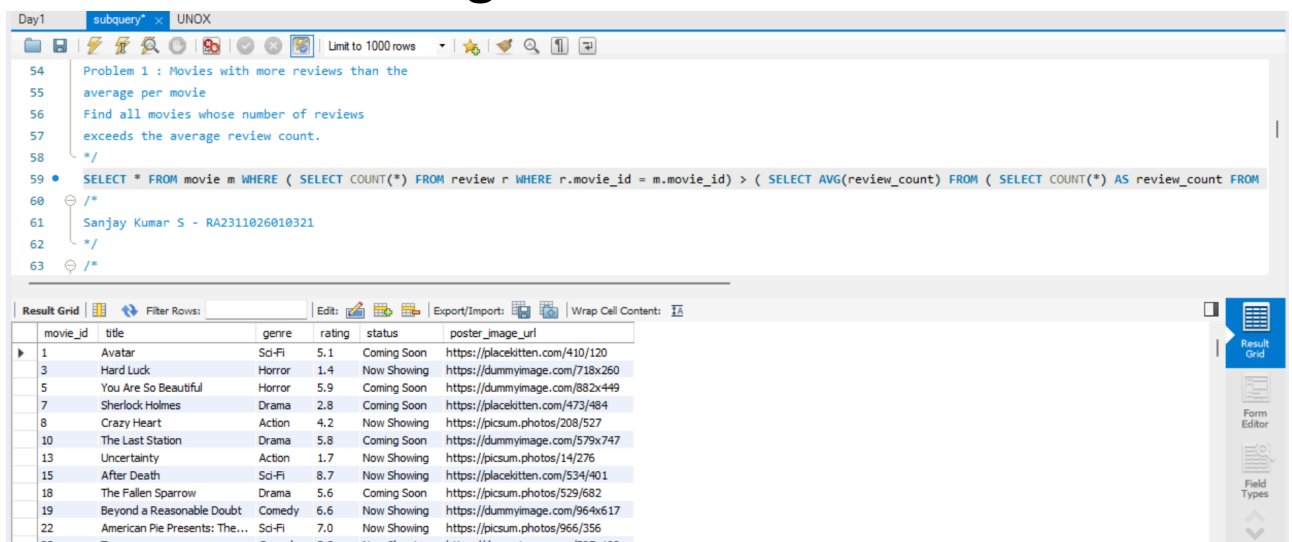
LEVEL 2 – Correlated & EXISTS Subqueries

Problem 1: Movies with more reviews than the average per movie

Complexity: **Easy**

Scenario:

Find all movies whose number of reviews exceeds the average review count.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```

54 Problem 1 : Movies with more reviews than the
55 average per movie
56 Find all movies whose number of reviews
57 exceeds the average review count.
58 */
59 • SELECT * FROM movie m WHERE ( SELECT COUNT(*) FROM review r WHERE r.movie_id = m.movie_id ) > ( SELECT AVG(review_count) FROM ( SELECT COUNT(*) AS review_count FROM
60 */
61 Sanjay Kumar S - RA2311026010321
62 */
63 */

```

The result grid displays the following data:

movie_id	title	genre	rating	status	poster_image_url
1	Avatar	Sci-Fi	5.1	Coming Soon	https://placekitten.com/410/120
3	Hard Luck	Horror	1.4	Now Showing	https://dummyimage.com/718x260
5	You Are So Beautiful	Horror	5.9	Coming Soon	https://dummyimage.com/882x449
7	Sherlock Holmes	Drama	2.8	Coming Soon	https://placekitten.com/473/484
8	Crazy Heart	Action	4.2	Now Showing	https://picsum.photos/208/527
10	The Last Station	Drama	5.8	Coming Soon	https://dummyimage.com/579x747
13	Uncertainty	Action	1.7	Now Showing	https://picsum.photos/14/276
15	After Death	Sci-Fi	8.7	Now Showing	https://placekitten.com/534/401
18	The Fallen Sparrow	Drama	5.6	Coming Soon	https://picsum.photos/529/682
19	Beyond a Reasonable Doubt	Comedy	6.6	Now Showing	https://dummyimage.com/964x617
22	American Pie Presents: The...	Sci-Fi	7.0	Now Showing	https://picsum.photos/966/356
23	Tessie	Comedy	6.9	Now Showing	https://dummyimage.com/1070x108

SQL Solution:

Problem 2: Movies that have at least one review

Complexity: **Easy**

Scenario:

Find all movies that have been reviewed by users at least once.

Day1 subquery UNOX

Limit to 1000 rows

```

63  /*
64  Problem 2 : Movies that have at least one review
65  Find all movies that have been reviewed
66  by users at least once.
67  */
68  • SELECT * FROM movie m WHERE EXISTS ( SELECT * FROM review r WHERE r.movie_id = m.movie_id );
69  /*
70  Sanjay Kumar S - RA2311026010321
71  */
72

```

Result Grid

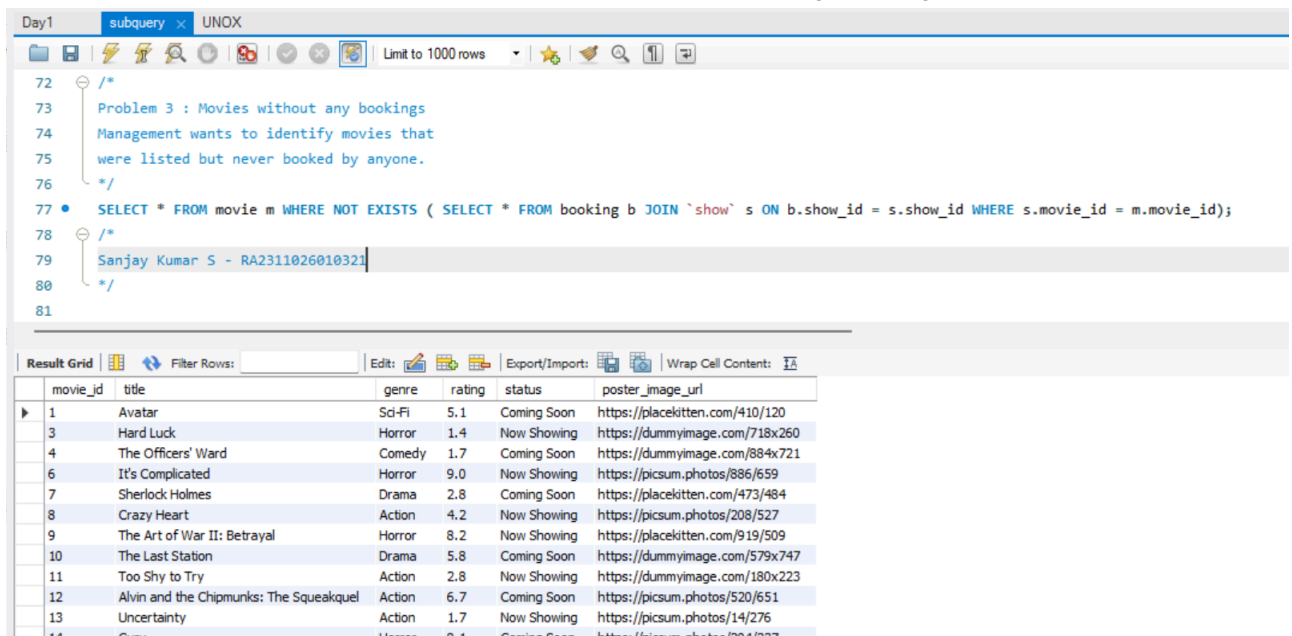
movie_id	title	genre	rating	status	poster_image_url
1	Avatar	Sci-Fi	5.1	Coming Soon	https://placekitten.com/410/120
2	Nine	Comedy	7.5	Now Showing	https://placekitten.com/80/766
3	Hard Luck	Horror	1.4	Now Showing	https://dummyimage.com/718x260
4	The Officers' Ward	Comedy	1.7	Coming Soon	https://dummyimage.com/884x721
5	You Are So Beautiful	Horror	5.9	Coming Soon	https://dummyimage.com/882x449
6	It's Complicated	Horror	9.0	Now Showing	https://picsum.photos/886/659
7	Sherlock Holmes	Drama	2.8	Coming Soon	https://placekitten.com/473/484
8	Crazy Heart	Action	4.2	Now Showing	https://picsum.photos/208/527
9	The Art of War II: Betrayal	Horror	8.2	Now Showing	https://placekitten.com/919/509
10	The Last Station	Drama	5.8	Coming Soon	https://dummyimage.com/579x747
11	Too Shy to Try	Action	2.8	Now Showing	https://dummyimage.com/180x223
12	Alvin and the Chipmunks	Action	6.7	Coming Soon	https://picsum.photos/520/451

Problem 3 (Higher-order): Movies without any bookings

Complexity: **Medium**

Scenario:

Management wants to identify movies that were listed but never booked by anyone.



The screenshot shows a SQL query editor window titled "Day1" with a tab "subquery" and "UNOX". The query is as follows:

```

72  /*
73  Problem 3 : Movies without any bookings
74  Management wants to identify movies that
75  were listed but never booked by anyone.
76  */
77  • SELECT * FROM movie m WHERE NOT EXISTS ( SELECT * FROM booking b JOIN `show` s ON b.show_id = s.show_id WHERE s.movie_id = m.movie_id);
78  /*
79  Sanjay Kumar S - RA2311026010321
80  */
81

```

Below the query editor is a "Result Grid" showing the results of the query. The grid has columns: movie_id, title, genre, rating, status, and poster_image_url. The results are as follows:

movie_id	title	genre	rating	status	poster_image_url
1	Avatar	Sci-Fi	5.1	Coming Soon	https://placekitten.com/410/120
3	Hard Luck	Horror	1.4	Now Showing	https://dummyimage.com/718x260
4	The Officers' Ward	Comedy	1.7	Coming Soon	https://dummyimage.com/884x721
6	It's Complicated	Horror	9.0	Now Showing	https://picsum.photos/886/659
7	Sherlock Holmes	Drama	2.8	Coming Soon	https://placekitten.com/473/484
8	Crazy Heart	Action	4.2	Now Showing	https://picsum.photos/208/527
9	The Art of War II: Betrayal	Horror	8.2	Now Showing	https://placekitten.com/919/509
10	The Last Station	Drama	5.8	Coming Soon	https://dummyimage.com/579x747
11	Too Shy to Try	Action	2.8	Now Showing	https://dummyimage.com/180x223
12	Alvin and the Chipmunks: The Squeakquel	Action	6.7	Coming Soon	https://picsum.photos/520/651
13	Uncertainty	Action	1.7	Now Showing	https://picsum.photos/14/276
14	Curtain	Horror	8.1	Coming Soon	https://picsum.photos/204/227

LEVEL 3 – Nested & Multi-Layer Subqueries

Problem 1: Users who booked highly-rated movies

Complexity: **Hard**

Scenario:

Find users who booked movies with a rating above 8.5.

The screenshot shows a SQL IDE interface with a query editor and a results grid. The query is a nested subquery designed to find users who have booked movies with a rating above 8.5.

```

81  /*
82  LEVEL 3 - Nested & Multi-Layer Subqueries
83  */
84  /*
85  Problem 1 : Users who booked highly-rated movies Find users who booked movies with ratings above 8.5.
86  */
87  SELECT * FROM user WHERE user_id IN ( SELECT b.user_id FROM booking b WHERE b.show_id IN ( SELECT show_id FROM `show` WHERE movie_id IN ( SELECT movie_id FROM movie
88  /*
89  Sanjay Kumar S - RA2311026010321
90  */

```

The results grid displays a list of users with their IDs, names, emails, and phone numbers. The first user listed is Lori Robinson with user_id 7.

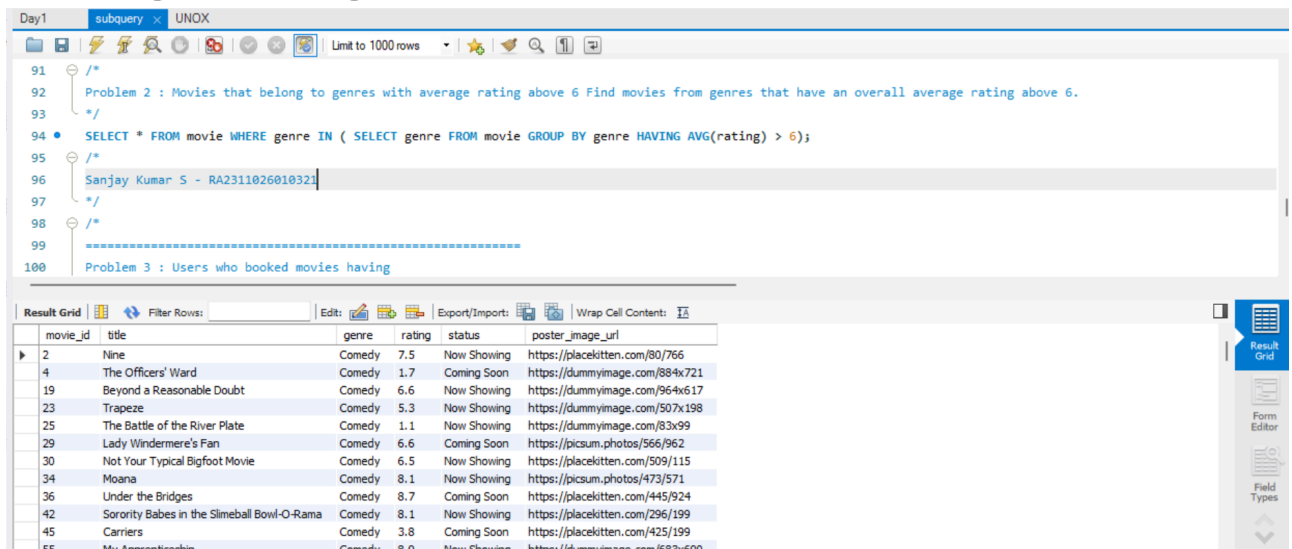
user_id	name	email	phone
7	Lori Robinson	david01@hotmail.com	4377597119
25	Martha Soto	powellkelly@outlook.com	5006748521
26	Christopher Nichols	melendezandrea@yahoo.com	2923384125
44	Alyssa Hunter	latoya85@hotmail.com	6835727715
48	Michelle Palmer	jsullivan@gmail.com	3536698968
49	Dawn Ali	hayesstephanie@gmail.com	8491010623
52	Danielle Fernandez	meagandaniels@hotmail.com	1154289823
53	Rick Cooper	shellybarker@gmail.com	7228042595
64	Brian Blankenship	rick01@yahoo.com	4746946539
72	Mary Smith	johnsonjohn@icloud.com	1438550436
73	Ashley Ford	qmartinez@gmail.com	5640596033

Problem 2: Movies that belong to genres with average rating above 6

Complexity: **Easy**

Scenario:

Find movies from genres that have an overall average rating above 6.



The screenshot shows a database query editor interface. The top section displays a SQL query for finding movies from genres with an average rating above 6. The query is as follows:

```

/*
Problem 2 : Movies that belong to genres with average rating above 6 Find movies from genres that have an overall average rating above 6.
*/
SELECT * FROM movie WHERE genre IN ( SELECT genre FROM movie GROUP BY genre HAVING AVG(rating) > 6);
/*
Sanjay Kumar S - RA2311026010321
*/
/*
=====
Problem 3 : Users who booked movies having

```

The bottom section shows the 'Result Grid' with the following data:

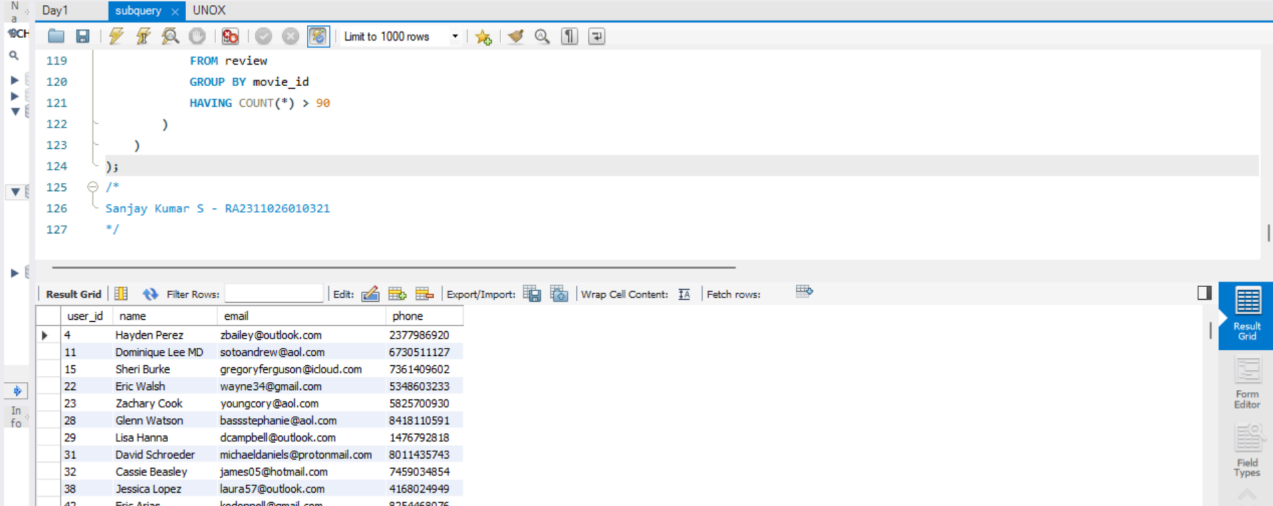
movie_id	title	genre	rating	status	poster_image_url
2	Nine	Comedy	7.5	Now Showing	https://placekitten.com/80/766
4	The Officers' Ward	Comedy	1.7	Coming Soon	https://dummyimage.com/884x721
19	Beyond a Reasonable Doubt	Comedy	6.6	Now Showing	https://dummyimage.com/964x617
23	Trapeze	Comedy	5.3	Now Showing	https://dummyimage.com/507x198
25	The Battle of the River Plate	Comedy	1.1	Now Showing	https://dummyimage.com/83x99
29	Lady Windermere's Fan	Comedy	6.6	Coming Soon	https://picsum.photos/566/962
30	Not Your Typical Bigfoot Movie	Comedy	6.5	Now Showing	https://placekitten.com/509/115
34	Moana	Comedy	8.1	Now Showing	https://picsum.photos/473/571
36	Under the Bridges	Comedy	8.7	Coming Soon	https://placekitten.com/445/924
42	Sorority Babes in the Slimeball Bowl-O-Rama	Comedy	8.1	Now Showing	https://placekitten.com/296/199
45	Carriers	Comedy	3.8	Coming Soon	https://placekitten.com/425/199
46	Mr. Armstrong	Comedy	8.0	Now Showing	https://dummyimage.com/602x600

Problem 3 (Higher-order): Users who booked movies having more than 90 reviews

Complexity: **Hard**

Scenario:

Find users who booked movies that have received more than 90 reviews.



The screenshot shows a SQL query editor with the following query:

```

119 FROM review
120 GROUP BY movie_id
121 HAVING COUNT(*) > 90
122 )
123 )
124 );
125 /*
126 Sanjay Kumar S - RA2311026010321
127 */

```

The result grid displays the following data:

user_id	name	email	phone
4	Hayden Perez	zbailey@outlook.com	2377986920
11	Dominique Lee MD	sotoandrew@aol.com	6730511127
15	Sheri Burke	gregoryferguson@icloud.com	7361409602
22	Eric Walsh	wayne34@gmail.com	5348603233
23	Zachary Cook	youngcory@aol.com	5825700930
28	Glenn Watson	bassstephanie@aol.com	8418110591
29	Lisa Hanna	dcampbell@outlook.com	1476792818
31	David Schroeder	michaeldaniels@protonmail.com	8011435743
32	Cassie Beasley	james05@hotmail.com	7459034854
38	Jessica Lopez	laura57@outlook.com	4168024949
42	Eric Arias	kodonnell@gmail.com	8254468076

